

E-Commerce Platform

Complete Documentation Suite · Spring Boot · Kafka · Kubernetes · Floci (AWS)

Generated: 17/7/2026

START HERE — Read This First

What is this project?

An online shop (like a tiny Amazon) built the way real companies build software:

6 small Java programs ("microservices") that work together, deployed with the same tools professionals use — Docker, Kubernetes, AWS (simulated), and automated pipelines.

The 6 services, in one line each

Service	Job	Port
user-service	Register, login, give out JWT tokens	8081
product-service	Show products (with a Redis cache for speed)	8082
order-service	Take orders, run the "order saga"	8083
inventory-service	Track stock, reserve it when someone orders	8084
payment-service	Charge the customer (simulated)	8085
notification-service	Send emails when things happen	8086

Dictionary — every scary word, in plain English

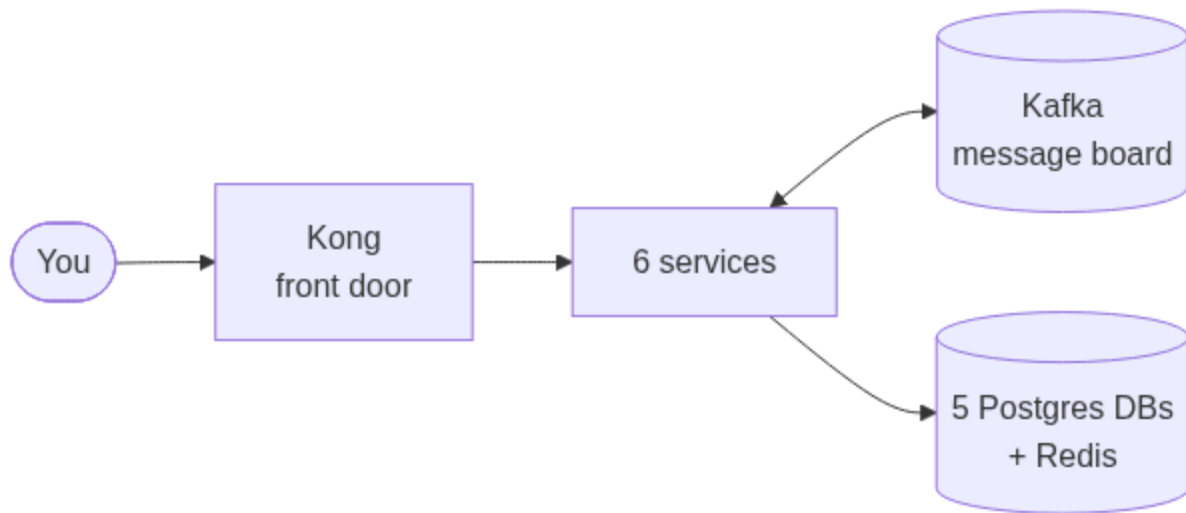
Word	Meaning
Container	One program packed with everything it needs, running in isolation. Like a food truck: kitchen + ingredients + menu, all in one box.

Docker	The tool that builds and runs containers.
Image	The frozen recipe/blueprint of a container. <code>docker build</code> makes an image; running it makes a container.
Pod	Kubernetes' word for "one running container that I manage."
Kubernetes (K8s)	The head office that manages hundreds of containers: restarts dead ones, adds more when busy. It never cooks — it only manages trucks.
Deployment	An order to Kubernetes: "keep N copies of this app alive, always." For apps that hold no data.
StatefulSet	Same, but for apps that DO hold data (databases). Each gets a permanent disk.
Service (K8s)	A fixed internal phone number. Callers dial the name; K8s connects them to any healthy pod.
Kafka	A message board between services. One service posts an event, others read it. Services never call each other directly.
Redis	A super-fast memory store. Used here for: login sessions, product cache, stock locks.
Kong	The API gateway — the single front door. Every request enters here first and gets routed to the right service.
AWS	Amazon's cloud — rent servers/databases instead of owning them.
Floci	A fake AWS running on your laptop. Same commands, zero cost.
CI/CD	Robots that test and deploy your code automatically every time you push to GitHub.
JWT	A signed ticket proving who you are. Login once, show the ticket on every request.

The learning order (do NOT skip ahead)

1. 01-DOCKER.md → how one service becomes a container
2. 02-KUBERNETES.md → how containers are managed at scale
3. 03-KAFKA.md → how services talk without calling each other
4. 04-AWS-FLOCI.md → what the cloud pieces are for
5. 05-CICD.md → how code goes from git push to running in production
6. 06-RUNBOOK.md → actually run everything + debug commands

The 30-second big picture



A request enters through Kong, hits one service, that service saves to its own

database and posts events on Kafka; other services react to those events. That's the whole system. Every doc after this just zooms into one part.

01 — Docker (How One Java App Becomes a Container)

The problem Docker solves

"It works on my machine" — your app needs Java 21, a specific Postgres, specific ports.

Docker packs the app **with everything it needs** into one box, so it runs identically on your laptop, your teammate's laptop, and the cloud.

The 3 words

```
Dockerfile → docker build → IMAGE → docker run → CONTAINER
(recipe)                (frozen meal)                (meal being eaten)
```

This project's Dockerfile (every service has the same one)

```
FROM eclipse-temurin:21-jre-alpine # start from a tiny Linux with Java 21
COPY target/*.jar app.jar          # put your compiled Spring Boot jar in
ENTRYPOINT ["java","-jar","app.jar"] # when the container starts, run the jar
```

Three lines. That's the whole recipe.

docker-compose.yml — run ALL 19 containers with one command

Instead of typing `docker run` 19 times, `docker-compose.yml` lists everything:

6 services + 5 Postgres + Redis + Kafka + Kong + Floci. One command starts the lot:

```
docker-compose up --build -d
```

The 3 patterns inside compose (this is 90% of the file)

1. Containers find each other BY NAME, never by localhost:

```
SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-user:5432/userdb
#                               ^^^^^^^^^^^^^^^^^ container name = hostname
```

Inside Docker's network, `localhost` means "this container itself" — a classic beginner bug. `postgres-user` is the name of the DB container, and Docker's internal DNS resolves it. **2. Ports map outside:inside:**

```
ports:
  - "5433:5432" # your laptop's 5433 → container's 5432
```

Five Postgres containers all listen on 5432 *inside*; they get different *outside* ports (5432-5436) so they don't clash on your machine. **3. Secrets come from `.env`, never hardcoded:**

```
POSTGRES_PASSWORD: ${USER_DB_PASS} # value lives in .env (gitignored)
```

Volumes — where data survives

```
volumes:
  - postgres_user_data:/var/lib/postgresql/data
```

A **named volume** = a disk Docker manages. Kill the container, data stays. Without it, every restart = empty database.

Try it

```
docker-compose up -d      # start everything
docker ps                 # list running containers
docker logs user-service  # read one service's logs
docker-compose down       # stop (data kept)
docker-compose down -v    # stop AND wipe data
```

Next: **02-KUBERNETES.md** — what happens when you need to run these containers across many machines with automatic healing and scaling.

02 — Kubernetes (The Head Office for Containers)

Start with the analogy — food trucks

- One **container** = one food truck (self-contained, sells one thing)
 - **Kubernetes** = the head office managing hundreds of trucks across a city:
- Truck breaks down → replacement sent automatically - One street gets busy → send more trucks there (autoscaling) - New menu rollout → swap trucks one at a time, street never empty
- A **pod** = head office's word for "one truck I manage" (in this project, 1 pod = 1 container)

Stateless vs Stateful — the most important split

	Stateless (cashier)	Stateful (record book)
Holds data itself?	No — data lives in the DB	Yes — IS the database
If it dies?	Start a fresh copy, nothing lost	Must reconnect to the SAME disk
K8s object	Deployment	StatefulSet + PersistentVolumeClaim
In this project	6 services, Kong	5 Postgres, Kafka, Redis

THE core workflow — what actually happens (memorize this one)

This loop **never stops**. It's called the reconciliation loop:



One sentence: you declare *what you want*; controllers loop forever fixing the difference between "want" and "have." That's why a killed pod resurrects itself.

Labels are the glue



Nothing references anything by ID — everything just **matches the same label**. Mismatch the label = everything silently disconnects (classic bug).

```
kind: Deployment                # stateless app
spec:
  replicas: 1                   # keep exactly 1 pod alive
  selector:
    matchLabels:
      app: user-service         # "find pods with this tag"
  template:                     # blueprint for new pods
    metadata:
      labels:
        app: user-service       # "stamp this tag on pods I create" (MUST match selector)
    spec:
      containers:
        - image: REGISTRY/ecommerce/user-service:IMAGE_TAG # placeholder, swapped at deploy
          env:
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:      # "fetch from the locked safe (Secret), not written here"
                  name: app-secrets
                  key: USER_DB_PASS
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:      # "fetch from the locked safe (Secret), not written here"
                  name: app-secrets
                  key: USER_DB_PASS
          resources:
            requests: {memory: 256Mi, cpu: 250m} # guaranteed minimum
            limits:   {memory: 512Mi, cpu: 500m} # hard ceiling
          livenessProbe: # "are you alive?" fail 3x → restart pod
            httpGet: {path: /actuator/health, port: 8081}
          readinessProbe: # "ready for traffic?" fail → pause traffic, don't kill
            httpGet: {path: /actuator/health, port: 8081}
```

Then a `Service` (fixed phone number) and an `HPA` (CPU > 70% → up to 3 pods).

All 6 services repeat this exact shape — learn one block, you know them all.

Commands you'll actually use

```
kubectl get pods -n ecommerce      # what's running?
kubectl logs <pod> -n ecommerce    # my app's logs
kubectl describe pod <pod> -n ecommerce # why is it broken? (events at bottom)
kubectl delete pod <pod> -n ecommerce # kill one — watch it come back!
```

Next: **03-KAFKA.md** — how these pods talk to each other without direct calls.

03 — Kafka (How Services Talk Without Calling Each Other)

The idea in one line

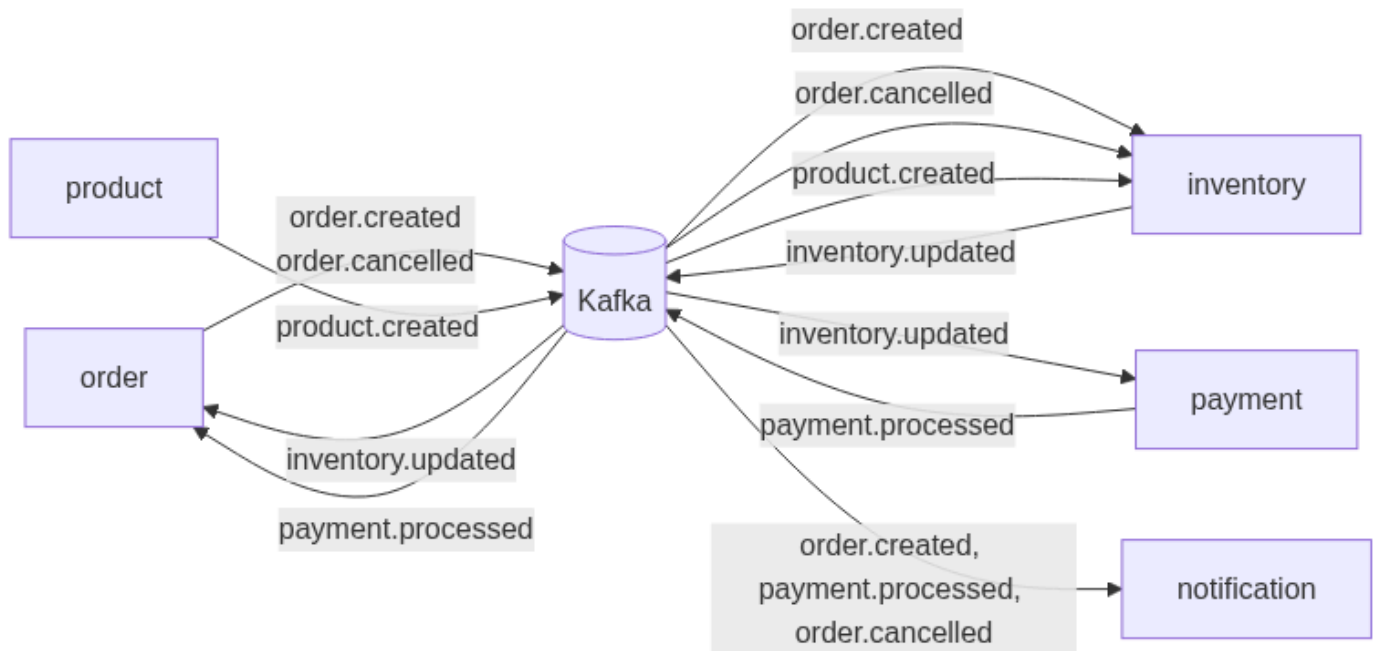
Services drop messages on a notice board (topic); whoever cares, reads them. Nobody waits for anybody. Nobody knows anybody's address.

Why not just call each other's REST APIs?

If order-service called payment-service directly and payment was down → order fails.

With Kafka: order posts "order created!" and moves on. Payment reads it whenever it's back up. **Loose coupling + no lost work.**

Who posts, who reads (this project)



5 topics: `order.created` `inventory.updated` `payment.processed` `order.cancelled` `product.created`

The Order Saga — the heart of this project

A "saga" = a multi-step business flow spread across services, driven by events:



Key design rule: payment fires ONLY after inventory confirms stock. Never charge a customer for something you can't ship.

The 4 words you need

Word	Meaning
Topic	One named notice board (e.g. <code>order.created</code>)
Partition	A topic's lane. More lanes = more parallel readers. Same key (orderId) → same lane → events stay in order per order
Consumer group	One service's team of readers. Each service has its OWN group, so every service gets its own full copy of every message
concurrency	Threads per pod reading lanes in parallel (never set above partition count)

Reliability settings (KafkaConfig.java)

```
ACKS_CONFIG = "all"           // message counts as sent only when safely stored
RETRIES_CONFIG = 3             // retry network blips
ENABLE_IDEMPOTENCE_CONFIG = true // a retry can never create a duplicate charge
```

Java side — how simple it actually is

Post a message:

```
kafkaTemplate.send("order.created", orderId.toString(), payloadJson);
```

Read messages:

```
@KafkaListener(topics = "order.created", groupId = "inventory-service-group", concurrency =
"6")
public void handleOrderCreated(String message) { ... }
```

That's it. Spring handles connections, retries, and group balancing.

Next: **04-AWS-FLOCI.md** — the cloud pieces around all this.

04 — AWS & Floci (The Cloud, Without the Bill)

What is AWS, really?

Renting instead of owning. Instead of buying servers, you rent: machines (EC2),

databases (RDS), file storage (S3), a Kubernetes brain (EKS)... per hour.

What is Floci?

A **fake AWS on your laptop**. It listens on `localhost:4566` and accepts the exact same commands as real AWS (`aws eks create-cluster` , `aws ecr ...`) — but instead of billing you, it starts local containers. Same learning, zero cost.

The pieces this project uses, and what each is FOR

Piece	Plain-English job	Created by
VPC	Your private fenced-off network in the cloud	<code>scripts/01-vpc.sh</code>
Subnets	Rooms inside the fence: <i>*public*</i> (internet can reach) and <i>*private*</i> (hidden)	<code>01-vpc.sh</code>
Security Groups	Per-door firewall rules ("only port 443 in")	<code>01-vpc.sh</code>
ALB	The front gate: receives ALL internet traffic, spreads it across Kong pods	<code>scripts/02-alb.sh</code>
WAF	Bouncer at the gate: blocks attack-shaped requests	<code>02-alb.sh</code>
Route 53	DNS — turns a name into your ALB's address	<code>02-alb.sh</code>
ECR	Shelf where your Docker images are stored; cluster pulls from here	<code>scripts/03-ecr.sh</code>
EKS	AWS-managed Kubernetes brain (you saw it in doc 02)	<code>scripts/04-eks.sh</code>
SES	Email-sending service (notification-service uses it)	used directly by code

How it fits together



Why public/private split? Databases and services must NEVER be reachable from the internet directly. Only the front gate (ALB) is public; everything else hides in private subnets. This is the #1 cloud security pattern.

The deploy scripts, in order

```
01-vpc.sh    → build the fenced network
02-alb.sh    → build the front gate (+ WAF + DNS)
03-ecr.sh    → build 6 images, push to the shelf
04-eks.sh    → create the Kubernetes cluster (Floci actually starts k3s)
05-deploy.sh → kubectl apply everything in the right order
06-teardown.sh → delete it all
```

05-deploy.sh order matters: namespace → secrets/config → databases (wait ready) → Kong → services → register Kong in ALB → monitoring. Rule: **a thing must exist before anything that references it starts.**

EKS vs ECS vs Fargate (interview one-liner)

- **EKS** = AWS-managed *Kubernetes* (this project)
- **ECS** = AWS's own non-Kubernetes container manager (different, simpler, AWS-only)
- **Fargate** = a *node option* under EKS/ECS: no visible machines, AWS finds capacity per-pod

(this project uses EC2 node groups instead — see `04-eks.sh`)

Next: **05-CICD.md** — how a `git push` turns into a tested, deployed release.

05 — CI/CD (From git push to Production)

The idea

CI (Continuous Integration): every push → robots build + test your code. **CD** (Continuous Delivery): if green → robots deploy it. Humans only approve prod.

The git workflow (how you and teammates work)



- Nobody pushes to `develop` / `main` directly (branch protection blocks it)
- A PR merges only when reviews pass AND all 6 service tests are green

This project's pipeline (`.github/workflows/ci-cd.yml`)



What each piece means

Piece	Plain English
Matrix test	Run the same test job 6 times in parallel, once per service
Trivy	Scans your Docker images for known security holes
k3d	A tiny throwaway Kubernetes cluster created inside the CI machine, destroyed after
Smoke test	A script that uses the app like a real user: register → add product → stock → order → check it becomes PAID. If the saga breaks, the build fails
Environment <code>dev</code>	Auto-deploys, no questions
Environment <code>production</code>	Pipeline PAUSES until a human approves in GitHub UI
GitHub Secrets	Passwords the pipeline needs (DB creds, JWT secret) — stored in repo Settings → Secrets, never in code

Where do passwords live in each environment? (loose coupling)

Environment	Secrets live in
 Docker Compose (laptop)	<code>.env</code> file (gitignored)
 Floci/K8s	K8s <code>Secret</code> object (<code>kubectl create secret</code>)
 CI	GitHub Actions Secrets

Same code, three secret sources — **no file ever contains a real password.**

Reading a failed pipeline (your actual job as a dev)

1. GitHub → Actions tab → click the red run
2. Click the red job → the red step
3. Read the LAST 30 lines of the log — the real error is almost always there
4. Common failures: a test broke (fix the test/code), image build failed

(Dockerfile/jar problem), smoke test failed (the saga broke — check service logs)

Next: **06-RUNBOOK.md** — commands to run and debug everything yourself.

06 — Runbook (Run It, Break It, Fix It)

Level 1 — Docker Compose (5 minutes, easiest)

```
# one-time: create .env in project root (see .env values in team vault)
docker-compose up --build -d
docker ps                                # expect ~19 containers
curl http://localhost:8000/api/products  # through Kong - should answer
```

Full user journey by hand:

```
# 1. register
curl -X POST http://localhost:8000/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Ali","email":"a@a.com","password":"password123"}'
# 2. create product
curl -X POST http://localhost:8000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Phone","price":499.99,"category":"electronics"}'
# 3. add stock
curl -X POST http://localhost:8000/api/inventory/1/restock \
  -H "Content-Type: application/json" -d '{"quantity":10}'
# 4. order it
curl -X POST http://localhost:8000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"userId":1,"shippingAddress":"Pune","items":[{"productId":1,"productName":"Phone","quantity":1,"price":499.99}]}'
# 5. a few seconds later - status should be PAID (the saga ran!)
curl http://localhost:8000/api/orders/1
```

Level 2 — Floci + Kubernetes (the full cloud simulation)

```
docker-compose up -d floci          # start fake AWS
bash scripts/01-vpc.sh               # network
bash scripts/02-alb.sh               # front gate
bash scripts/03-ecr.sh               # build+push images
bash scripts/04-eks.sh               # kubernetes cluster
bash scripts/05-deploy.sh            # deploy everything
kubectl get pods -n ecommerce        # all should be Running
```

Level 3 — CI/CD

```
git push origin develop             # → tests + auto deploy-dev + smoke test
# PR develop→main → merge → approve the production gate in GitHub UI
```

Debug cheat-sheet (the commands that solve 90% of problems)

Symptom	Command	What to look for
Pod not starting	<code>kubectl describe pod</code> <code>-n ecommerce</code>	Events at the bottom: ImagePullBackOff? CreateContainerConfigError (missing Secret)?
App crashing	<code>kubectl logs</code> <code>-n ecommerce</code>	Java stack trace — usually DB not reachable or missing env var
Pod restarts repeatedly	<code>kubectl get pods -n ecommerce</code> (RESTARTS column)	livenessProbe failing → app too slow to start or /health broken
Order stuck PENDING	<code>kubectl logs -l app=inventory-service -n ecommerce</code>	Did inventory consume order.created? Stock available?
404 through Kong	<code>curl localhost:8001/routes</code>	Is the route there? <code>strip_path: false</code> present?
Kafka silence	exec into kafka pod → <code>kafka-topics --list --bootstrap-server localhost:9092</code>	Topics exist?
Everything weird	<code>docker-compose down -v && docker-compose up --build -d</code>	Nuclear option (wipes data)

Golden rules

1. **Logs first.** 90% of answers are in `kubectl logs` / `docker logs`.
2. **Events second.** `kubectl describe pod` events explain why K8s won't start something.
3. Container can't see `localhost` — services reach each other **by name**.
4. Killed a pod and it came back? That's not a bug — that's the reconciliation loop (doc 02).